

Mini-projet : Développement d'une application web complète avec Docker Compose

Titre : Application Web "Task Manager" avec Frontend, Backend et Base de Données

Objectifs

1. Apprendre à utiliser Docker Compose pour déployer une application web complète.
 2. Gérer l'intégration de plusieurs services : un frontend (React), un backend (Node.js/Express) et une base de données (PostgreSQL).
 3. Mettre en œuvre des volumes pour persister les données et configurer les ports pour une interaction avec les services.
-

Description du projet

L'application "Task Manager" permet aux utilisateurs de créer, lire, mettre à jour et supprimer des tâches (CRUD). L'architecture comprend :

- **Frontend** : Une interface utilisateur construite avec React pour gérer les tâches.
 - **Backend** : Une API REST en Node.js (Express) pour gérer les données.
 - **Base de données** : PostgreSQL pour stocker les tâches.
-

Structure du projet

```
task-manager/  
├── backend/  
│   ├── Dockerfile  
│   ├── package.json  
│   └── server.js  
├── frontend/  
│   ├── Dockerfile  
│   ├── package.json  
│   └── src/  
│       └── App.js  
└── docker-compose.yaml
```

Étape 1 : Backend (Node.js/Express)

Créer le fichier backend/Dockerfile :

```
FROM node:14
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Créer le fichier backend/package.json :

```
json
{
  "name": "task-manager-backend",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "express": "^4.17.1",
    "pg": "^8.7.3"
  }
}
```

1.

Créer le fichier backend/server.js :

```
const express = require("express");
const { Pool } = require("pg");
const app = express();
const port = 5000;

app.use(express.json());

const pool = new Pool({
  user: "postgres",
  host: "db",
  database: "tasks",
  password: "password",
  port: 5432,
});

app.get("/tasks", async (req, res) => {
  const result = await pool.query("SELECT * FROM tasks");
```

```
        res.json(result.rows);
    });

app.post("/tasks", async (req, res) => {
    const { name } = req.body;
    await pool.query("INSERT INTO tasks (name) VALUES ($1)",
    [name]);
    res.send("Task added!");
});

app.listen(port, () => {
    console.log(`Backend running on port ${port}`);
});
```

Étape 2 : Frontend (React)

Créer le fichier **frontend/Dockerfile** :

```
FROM node:14
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Créer le fichier **frontend/package.json** :

```
{
  "name": "task-manager-frontend",
  "version": "1.0.0",
  "dependencies": {
    "react": "^17.0.2",
    "react-dom": "^17.0.2",
    "axios": "^0.21.1"
  }
}
```

1.

Créer le fichier **frontend/src/App.js** :

```
import React, { useState, useEffect } from "react";
import axios from "axios";

function App() {
  const [tasks, setTasks] = useState([]);
  const [task, setTask] = useState("");

  useEffect(() => {
    axios.get("http://localhost:5000/tasks").then((response) => {
      setTasks(response.data);
    });
  }, []);

  const addTask = () => {
    axios.post("http://localhost:5000/tasks", { name: task })
      .then(() => {
        setTask("");
        window.location.reload();
      });
  };

  return (
    <div>
      <h1>Task Manager</h1>
      <input
        value={task}
        onChange={(e) => setTask(e.target.value)}
        placeholder="Add a task"
      />
      <button onClick={addTask}>Add</button>
      <ul>
        {tasks.map((t, index) => (
          <li key={index}>{t.name}</li>
        ))}
      </ul>
    </div>
  );
}
```

```
export default App;
```

Étape 3 : Base de données (PostgreSQL)

Le service PostgreSQL sera configuré via Docker Compose.

Étape 4 : Fichier `docker-compose.yaml`

Créer le fichier `docker-compose.yaml` à la racine du projet :

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - db

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

  db:
    image: postgres:13
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: tasks
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Étape 5 : Lancer le projet

Démarrez les services :

```
docker-compose up -d
```

1. Accédez aux services :
 - **Frontend** : <http://localhost:3000>
 - **Backend** : <http://localhost:5000>
2. Interagissez avec l'application via l'interface React.

Étape 6 : Arrêt et nettoyage

Pour arrêter et supprimer les conteneurs :

```
docker-compose down
```